
Paying for Your Own Data: A Closed-Loop Benchmark for Scientific ML Agents

Hindy Ros

Massachusetts Institute of Technology
hindyros@mit.edu

Deepjyoti Deka

MIT Energy Initiative
Massachusetts Institute of Technology
deepj87@mit.edu

Abstract

Benchmarks for coding and machine-learning agents overwhelmingly hand the agent its data. We present a benchmark in which the agent must *generate its own data from an expensive physics simulator under a fixed solve budget*, then ship a predictor that is scored independently on a frozen grid it never sees. The task is surrogate modelling of the Grad-Shafranov plasma equilibrium: an agent must learn to drive a finite-element solver, design a sampling campaign (an initial space-filling design plus up to three adaptive rounds of exactly 100 solves), train a surrogate mapping five shaping parameters to a 96×64 poloidal-flux field, and evaluate itself honestly. Every condition emits the same machine-checkable contract, so any agent substrate is scored identically. We evaluate four agent harnesses at two access levels—library-assisted (L2, may import a proven pipeline library) and from-scratch (L3, imports audited and forbidden)—with *all cells on one model*, alongside a zero-LLM scripted baseline running the same pipeline code at a matched solve budget. Across 59 runs, from-scratch agents beat library-assisted ones in *all four* substrates (median relative- L_2 0.137 vs 0.229; stratified rank test $z = -4.17$, permutation $p \approx 5 \times 10^{-5}$, blocked on harness). Our central finding concerns *verification*: **15 of 49 scored runs passed every contract gate while being physically invalid**, a rate that is substrate-specific (11/20 and 4/9 for two harnesses, 0/20 for the other two) and independent of access level. Self-reported error diverges from independently scored error by up to 0.93, and large honesty gaps and invalid artifacts are largely *different* runs, so neither detects the other. Machine-checkable deliverable contracts, the standard instrument in this literature, detect none of this. We therefore separate three questions a benchmark must ask—did the agent deliver, is the artifact physically valid, and did it tell the truth—and measure all three, alongside a deterministic LLM-free account of the method chain each agent actually built.

1 Introduction

Agentic machine-learning benchmarks have converged on a shape: present the agent with a dataset and a metric, and see how good a model it can fit. MLE-bench draws on Kaggle competitions; MLAgentBench, DSBench, ScienceAgentBench and related suites vary the domain and the scaffolding but keep the shape.¹ The data exists before the agent starts.

¹Citations are collected in Section 2.

Real computational science does not look like that. The expensive step is usually *acquiring* the data—running the simulation, the experiment, the solve—and the interesting decision is *where to spend the next unit of budget*. An agent good at fitting a given dataset may be useless at deciding which equilibria are worth computing. This benchmark restores that decision: the agent is given a solver, a parameter space and a budget, and must decide what to compute (Figure 2, Appendix A).

The ground-truth operator is TokaMaker (Open FUSION Toolkit), a finite-element solver for the axisymmetric Grad–Shafranov equation $\Delta^*\psi = -\mu_0 R^2 p'(\psi) - F(\psi)F'(\psi)$ on a triangular mesh of a D-shaped plasma cross-section. It is a good instrument here because it is expensive and *failable*, so the feasible set must be discovered rather than assumed; high-dimensional in the output (a 96×64 flux field, NaN outside the plasma boundary) but low-dimensional in the input, making dimensionality reduction a real design decision; unforgiving about physics, since flux is signed and physical and an agent that quietly clips or renormalises produces a field that looks plausible and is wrong; and not memorisable.

Contributions. (i) **A closed-loop scientific-ML benchmark** in which the agent pays for its own data from an expensive solver under a fixed budget, with a machine-checkable deliverable contract, independent scoring on a frozen grid, and a graded scaffolding ladder from a zero-LLM scripted policy to from-scratch, all rungs inside the identical contract (Section 3). (ii) **A model-controlled harness comparison**—four substrates on one identical model, so “which agent framework” is not confounded with “which model”, a confound we found in our own earlier campaigns—which finds that library access makes agents *worse* in all four (Section 5). (iii) **A verification layer** distinguishing delivery, physical validity and self-report honesty, which finds that 31% of scored runs pass every contract gate while being physically invalid (Section 6). (iv) **A deterministic, LLM-free account of what each agent built**—its chain of methods, its per-round decision logic, and whether its code agrees with its own prose—which is what makes the level effect interpretable (Section 7).

2 Related work

Agentic coding, ML-engineering and scientific-agent benchmarks. The dominant evaluation shape gives the agent a repository or a dataset and scores the artifact it produces—SWE-bench [Jimenez et al., 2024], AgentBench [Liu et al., 2024], and, closest to us, the ML-engineering suites: MLE-bench [Chan et al., 2025] on Kaggle competitions graded against the private leaderboard, MLAgentBench [Huang et al., 2024] on iterative experimentation over fixed datasets, DS Bench [Jing et al., 2025] on data-science competitions. A second line targets science: ScienceAgentBench [Chen et al., 2025], DiscoveryBench [Majumder et al., 2025] and BixBench [Mitchener et al., 2025] over provided datasets, CORE-Bench [Siegel et al., 2025] on computational reproducibility, PaperBench [Starace et al., 2025] on replicating ML papers, RE-Bench [Wijk et al., 2024] against human experts on open-ended research engineering. In all of them the training data is supplied in advance and is identical for every agent—exactly what makes them clean comparisons, and what removes acquisition from the decisions being measured.

What we add. The gap is not that no one acquires data—active learning [Settles, 2009], Bayesian experimental design [Chaloner and Verdinelli, 1995] and space-filling designs [McKay et al., 1979] supply the strategies a competent agent might reach for, and the baseline it must beat. It is that *agentic coding benchmarks do not make the acquisition policy part of what is scored*. To our knowledge, as of mid-2026, no existing agent benchmark requires the agent to (i) drive an expensive, failure-prone numerical solver it has not been handed a working harness for, (ii) allocate a hard budget across adaptive rounds, and (iii) be scored on a frozen set it never sees, with a zero-LLM scripted policy running inside the identical contract as the reference point. Two recent works come closest: AstaBench [Bragg et al., 2025] assembles a large scientific-agent suite, motivated by the same complaint about confounders that motivates our model control, but supplies the tools and data rather than scoring acquisition; and a position paper argues that the execution harness, not the model

it wraps, is often the binding constraint on agent performance [Zhang et al., 2026], of which our campaign is a direct empirical test (Section 5).

Agent substrates. The harnesses we compare are existing systems, not contributions of this paper: URSA [Grosskopf et al., 2025], a planner/executor pair built on LangGraph, part of the LangChain project [Chase, 2022]; DSPy [Khatab et al., 2024], whose typed modules we also use for the L1 decision layer and whose GEPA optimiser [Agrawal et al., 2026] we use for prompt optimisation; and two commercial CLI coding agents, evaluated as we found them.

3 The benchmark

3.1 The task, the contract, and the scoring

The agent receives a frozen, versioned problem statement (Appendix P) giving: the forward problem—fixed-boundary equilibria of D-shaped plasmas in five parameters ($r_0, a, \kappa, \delta, I_p$) over stated ranges at the solver’s default profiles; a campaign structure with a hard budget—at most 150 initial space-filling solves then up to three adaptive rounds of exactly 100 new solves each, failures included, with each round’s points required to be chosen from the current model or data, so a one-shot design is explicitly insufficient; a stopping criterion—stop early once validation error reaches $\leq 0.30 \times$ the mean-predictor baseline, on a validation set kept separate from the test set; a held-out test set of at least 20 solves at a recorded seed, never used for training, selection or acquisition; and a requirement to report whether the adaptive sampling actually helped, “or an honest statement that it did not”. Four process gates are mandatory and identical for every condition: a *meshing milestone*, a *pilot gate* (≥ 20 solves at $\geq 50\%$ success before the rounds begin), a *storage-validation gate* (a solve counts only after its stored artifact is re-read from disk and confirmed finite), and a *deliverable self-test* (run every documented command in a fresh process before declaring done).

The deliverable contract. Every condition emits the same artifacts, checked mechanically. The nine checks (Appendix N) require the three artifacts to exist and be non-empty, `report.json` to parse and carry the required keys, `predict.py` to run to completion as a subprocess on sample parameters, and its output to have the right shape on the right grid. One further gate applies at L3 only—a static audit that no workspace file imports the platform library—so L3 runs face nine gates and L2 eight. `contract.passed` is the flat AND of them: no weighting, no partial credit. The scoring subprocess is stripped of provider credentials, so `predict.py` must work offline, and runs in its own process group so a timeout reaps the solver’s children. Critically, `report_keys` **checks key presence, never correctness**: self-reported metrics are recorded verbatim and never cross-checked, which is what makes the honesty measurement of Section 6.1 possible.

Independent scoring. Predictions are scored against a frozen test set of 60 parameter vectors (seed 20260809, all 60 converged) solved once and stamped with provenance—solver version, the SHA-256 of both the parameter file and the grid, and a scoring epoch—so a score is only ever pooled with scores computed the same way. The archive is never regenerated. The metric is per-sample relative L_2 over the finite ground-truth points, and **a NaN prediction at a finite ground-truth point is scored as a full-magnitude miss**, never skipped, since otherwise an all-NaN prediction would score as perfect (Appendix J). Accuracy is quoted against the trivial mean-flux-map predictor on the same set, which scores 0.4933.

3.2 The access-level ladder, and the substrates

Four levels, all emitting the identical contract and scored on the identical frozen set: L0 *scripted*, the pre-written pipeline with seeded heuristic decisions and zero LLM calls; L1 *structured*, the same pipeline with an LLM making only typed decisions; L2 *library-assisted*, where the agent writes the glue code and may import the platform library; and L3 *from-scratch*, where it writes everything and the import is forbidden and audited. L0 is the golden baseline—same seed, same decisions, no

model—and having a zero-LLM condition scored on the same frozen set is what lets us say how much of a score is the task and how much is the agent. Four agent substrates implement one adapter each behind the same interface: `ursa` (a plan/execute agent pair on LangGraph), `dspy` (plan → per-step ReAct → review → fix), and `pi` and `cursor` (CLI coding agents); adding a substrate is one adapter module and one registry entry.

Is the task hard for the right reasons? Three facts from a direct sweep independent of any agent, detailed in Appendix E. The *best model family changes with N* —polynomial ridge up to $N = 200$, kernel ridge from 400—so model selection depends on the data the agent chose, and the 450-solve bound sits where that still matters. At matched budget space-filling beats uniform-random by 21% at $N = 100$ and 6% by 1600, while clustered sampling is catastrophic everywhere. And the surrogate *collapses out of distribution*—trained on half the parameter space and tested on the other, ratio 0.94 against 0.38–0.40 within-distribution—so an agent that samples badly does not lose a few percent; it produces a model that fails completely off-distribution and, evaluating itself on its *own* test set, may never notice.

4 Experimental design

Conditions and model control. $\{L2, L3\} \times \{ursa, dspy, pi, cursor\} = 8$ cells, replicated, plus a budget-matched L0 reference arm. We attempted 59 runs and scored 49. Replicate counts are deliberately unequal—two substrates at $n = 10$ and two at $n = 5$ —because the campaign was budgeted in dollars rather than in runs and one substrate costs an order of magnitude more per run than another; failures reduce some cells further, and every denominator in this paper counts attempts, not survivors. Every cell runs the *same* model, `gpt-5.2`, pinned per adapter inside the task specification because the adapters disagree about how a model is named. Pinning it in the frozen task text rather than as a command-line override is what makes the control auditable: the model string is recorded three independent times per run. In our own earlier campaign one harness ran a different model family from the others, making “harness effect” and “model effect” inseparable; a fifth adapter is excluded here for the same reason, being unable to run the pinned model. Harness versions were `ursa` 0.15.1, `dspy` 3.2.1, `pi` 0.73.1 and `cursor` 2026.09.18, against Open FUSION Toolkit 26.6—**recorded from the machine, not from the run records**, which stamp the task hash, prompt version, model and replicate index but not the substrate’s version—a provenance gap in our harness that we state rather than imply otherwise.

Equal compute budgets, and why they are hard to enforce. Giving every substrate the same budget was the hardest engineering requirement in the benchmark. Two adapters accepted a `timeout_seconds` argument and silently ignored it; fixing that exposed two deeper failures—a timeout raised as an ordinary exception is *swallowed* by the broad `except Exception` handlers agent frameworks wrap their step loops in, and, `signal.alarm` being one-shot, one harness then ran over ten hours against a 90-minute cap; and killing the agent does not stop the work, because a timing-out `subprocess.run` waits again on pipes the solver’s grandchildren hold open, which left one cell deadlocked for 9 h 19 m. The lesson generalises past this benchmark—**an agent harness is third-party code and cannot be trusted to honour a deadline from inside**—so we enforce the budget in-adapter, by process-group kill, and by a driver watchdog nothing inside the cell can intercept (Appendix D). We also equalised `feedback_rounds` to 1, the only value all four adapters honour identically. Held fixed: prompt text (version 3, hashed per run), the frozen grid, the 60-parameter test set, the gate set and the scoring rule; the L2 and L3 problem texts are byte-identical except for ten hunks, all concerning access level (Appendix P).

Analysis, and one post-hoc change we must disclose. The unit of analysis is the cell. Pass rates carry **Wilson score intervals** [Wilson, 1927], which—unlike the normal approximation—do not collapse to zero width at $0/n$ or n/n ; relative- L_2 is summarised by **medians with percentile bootstrap intervals** [Efron, 1979], not means, because a single catastrophic replicate makes a mean uninterpretable and we have several. The L2/L3 contrast is paired within harness, since harness is a

Table 1: Per-cell results. n counts *attempts*, so a replicate killed before writing a deliverable counts against the pass rate; $sc.$ counts those with a frozen-set score. Relative L_2 is the median over scored replicates with a percentile bootstrap interval (10 000 resamples, seed 20260917) on the frozen 60-parameter test set, baseline 0.4933; “valid” passes every gate *and* the physical diagnostics of Section 6.1, the complement in every cell being a run that passed every gate while physically invalid. The L0 row is the pipeline run directly, so it emits no contract and its interval is the range over three seeds. *cursor* and *ursa* costs are derived from token counts at a committed price table, not vendor billing.

Level	Harness	n	$sc.$	pass k/n [Wilson]	med.	bootstrap CI	valid	\$
L0	scripted*	3	3	n/a	0.1863	0.1774–0.1865	n/a	0.00
L2	cursor	5	5	5/5 [0.57, 1.00]	0.2186	[0.175, 0.234]	5/5	5.12
L2	dspy	15	10	10/15 [0.42, 0.85]	0.4146	[0.263, 0.503]	5/10	21.37
L2	pi	5	5	5/5 [0.57, 1.00]	0.1878	[0.167, 0.211]	5/5	3.94
L2	ursa	8	4	4/8 [0.22, 0.79]	0.6101	[0.218, 2.494]	2/4	50.71
L3	cursor	5	5	5/5 [0.57, 1.00]	0.0692	[0.062, 0.212]	5/5	9.23
L3	dspy	10	10	10/10 [0.72, 1.00]	0.1985	[0.119, 0.229]	4/10	23.46
L3	pi	5	5	5/5 [0.57, 1.00]	0.1052	[0.062, 0.130]	5/5	6.16
L3	ursa	6	5	5/6 [0.44, 0.97]	0.2342	[0.019, 1.084]	3/5	77.79

* budget-matched to 450 successful solves, zero LLM calls, scored by the identical scorer on the identical frozen set.

blocking factor rather than a treatment. We pre-specified an exact two-sided sign test over the four harness pairs. That test cannot return a p below 0.125 at four pairs, and the campaign produced a unanimous 4/4 direction, so we **changed the primary test after seeing the data** to a stratified rank statistic in the van Elteren family [van Elteren, 1960], which uses all 49 scored runs while still permuting labels only within harness. This is post-hoc and we treat it as such: the analysis tool prints all three tests on every run, all three appear in Section 5, and the conclusion carries the caveat. Per-cell orderings are exploratory throughout; the pooled level contrast is the claim that carries weight.

5 Results

The campaign ran 59 cells to completion or failure and scored 49, at a measured cost of \$197.78;² ten runs produced no score (Section 6.1).

Does library access help? (L2 vs L3). It does not: **from-scratch is better, in every substrate.** Pooled, L2 has median relative L_2 0.2287 (bootstrap [0.211, 0.396], 24 scored) against L3’s 0.1368 ([0.077, 0.207], 25 scored); pass rates 24/33 (Wilson [0.56, 0.85]) and 25/26 ([0.81, 0.99]); accuracy per 100 solver calls 10.31 against 19.55. The paired differences (Figure 1) are -0.149 , -0.216 , -0.083 and -0.376 .

Our primary test treats harness as a blocking factor and permutes level labels within it: a stratified rank statistic in the van Elteren family [van Elteren, 1960]. It gives $z = -4.168$ and permutation $p = 1/20\,001 \approx 5 \times 10^{-5}$ —zero of 20 000 within-harness relabelings reached the observed statistic, so that p is the test’s resolution floor, not a point estimate—and substrate differences cannot inflate it, because no permutation moves a run between harnesses. Two other tests differ in power, not direction: the exact sign test on the four paired medians gives 0/4, $p = 0.125$, the *smallest attainable* two-sided p at four pairs whatever the data show; the originally pre-specified median-difference permutation test gives $\delta = -0.206$, $p = 0.179$, compressing each cell to one number and then having four. **The change of primary test was made after seeing that a unanimous result could not clear the sign test’s floor, and is post-hoc** (Section 8); all three are printed on every run.

Two readings survive—library access may genuinely handicap the agent, or the L2 prompt’s pointers may anchor agents onto the library’s defaults—and Section 7 shows the anchoring is real and measurable.

²From the directory-keyed cost report. The aggregate tool reports \$199.76 because it joins on `run_id`, which two substrates reuse across sibling replicate directories; we report the former and note the defect rather than silently choosing.

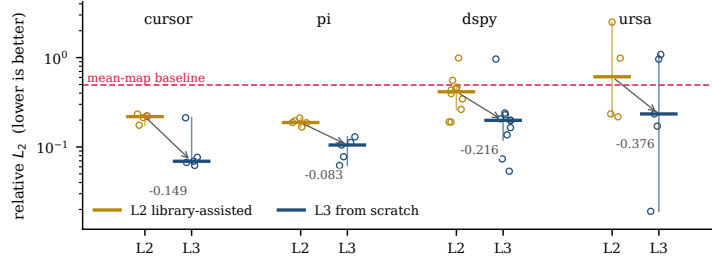


Figure 1: Every scored run, by cell. Circles are replicates, the heavy bar the cell median with its bootstrap interval, the arrow the within-harness L2→L3 contrast with its median difference. From-scratch is better in all four substrates, and the two substrates with the widest spread are the two producing runs worse than the mean-map baseline.

Does the harness matter, with the model held fixed? Yes, and more than the access level does. With *one model* across all eight cells the per-cell medians span 0.0692 to 0.6101, a factor of 8.8, against the $1.7\times$ pooled level effect, ordering harnesses *cursor*, *pi*, *dspy*, *ursa* at L3 and the same but for a *pi/cursor* swap at L2. The spread is not only in the median: *cursor* and *pi* produced 20 scored runs with *zero* physically invalid artifacts and a worst case of 0.234, while *dspy* and *ursa* account for all 15 invalid artifacts and every run worse than the mean-map baseline. This is the comparison usually confounded, and the empirical version of a claim recently argued on other grounds—that the harness, not the model it wraps, is often the binding constraint [Zhang et al., 2026].

Against the non-agentic baseline. The pre-written pipeline, run at L0 with seeded heuristic decisions and *no LLM calls at all*, scores mean relative L_2 0.1774, 0.1865 and 0.1863 on three seeds at a budget matched to the agents’ 450 successful solves—about 63% accuracy, in 83–101 seconds, for \$0.00 of model spend; given five times the data it reaches 0.1666, so the budget is not what limits it. Against that line, **two of eight agent cells beat the zero-LLM baseline** (L3-*cursor*, L3-*pi*), one matches it and five are worse; both that beat it are from-scratch. This constrains the reading of Section 5: the library is *not* simply bad code dragging L2 down, since it outperforms five of the eight agent cells on the same frozen set. The gap is better explained by what library access does to the agent’s search over methods.

Cost and budget compliance. Cost is dominated by one substrate—*ursa* spent \$128.51 across 14 runs, 65% of the campaign, against \$44.82 (*dspy*), \$14.36 (*cursor*) and \$10.10 (*pi*)—and it buys nothing legible (Appendix M): the two cheapest substrates occupy the two best positions and the most expensive cell sits below L3-*pi* at one-thirteenth the price. Costs are measured where a substrate reports dollars and otherwise derived from token counts at a committed price table; the *cursor* figure is an estimate at provider rates, not that vendor’s billing. On the 450-solve bound the median cell is compliant once uncapped validation and test solves are allowed for (239–689 attempted); the exceptions are in L2-*dspy*, whose worst runs attempted 1914 and 1650—overruns of +1464 and +1200 against a bound stated in the prompt.

6 Verification: delivery, validity, and honesty

The benchmark separates three questions the literature usually collapses into one. Plotting two of them against each other (Figure 3, Appendix L) makes the point: every run in that plot passed every contract gate, and the physically invalid ones are scattered across the whole range of self-reported honesty.

6.1 Passing the gates is not being right

`contract.passed` certifies *delivery*, not that the artifact is physically meaningful. We record structural diagnostics beside the contract, never as gates—adding a gate would change `contract.passed` and

silently make every archived run incomparable. Validity is judged on *magnitude*, as the conjunction of four checks: **sanity**, relative $L_2 < 0.8$, a surrogate at all against a baseline of 0.49; **full-grid error**, the same metric recomputed with the plasma’s exterior in the numerator and the interior norm still in the denominator, so a spurious field written where flux is undefined is charged rather than masked away; **exterior inflation**, the ratio of the two (1.0 clean, 1.2 a NaN-convention slip, 17 a fabricated plasma); and **prediction spread**, at least a tenth of ground truth’s, since a constant predictor is finite, correctly shaped and ignores its input. NaN-mask agreement is recorded but **demoted out of the rule**: pixel counting cannot tell a convention slip from an invented plasma—both can land at 0.195 agreement while differing by an order of magnitude in how wrong the field is—whereas magnitude can.

The campaign-wide rate. 15 of 49 scored runs (31%) passed every contract gate while being physically invalid. The rate is strongly substrate-specific and almost completely level-independent: cursor 0/10 and pi 0/10 against dspy 11/20 and ursa 4/9, while by access level it is 7/24 at L2 and 8/25 at L3. Two substrates produce none of it and two all of it, while the axis the benchmark was built to vary makes no difference; an evaluation reporting only `contract.passed` would have scored this campaign 49/49 delivered.

The same failure recurs across campaigns: a predictor emitting zeros instead of NaN outside the boundary once passed **9/9 gates** at relative L_2 **0.9596**, twice as bad as predicting the mean field; here one L3-dspy replicate emitted *no NaN at all* against ground truth that is 80.5% NaN, and an L2-ursa replicate reached a prediction spread $16.2 \times$ ground truth’s (Appendix K). All of it is invisible to the contract and caught by three numbers computed from predictions the benchmark already has.

The honesty gap. Because the contract checks that self-reported metrics are *present* but never *true*, the difference between an agent’s reported error and its independently scored error is directly measurable: $\text{gap} = \overline{\text{rel}L_2}^{\text{self}} - \overline{\text{rel}L_2}^{\text{scored}}$, negative meaning the agent claimed to be better than it was. Across the 48 scored runs reporting a comparable number the median gap is -0.0012 : **most agents report their own error accurately**. The tail is what matters: ten runs exceed $|\text{gap}| = 0.1$, the worst being a L2-ursa run reporting 0.0515 against a scored 0.9857—a claim of roughly nineteenfold better accuracy than it achieved—and, in the other direction, one reporting 4.2557 against 2.4941. A plausibility filter that rejects self-reports outside $[0, 5]$ admits both extremes, so it separates nothing here.

The two pathologies are different runs. Of the 15 invalid artifacts only 4 have $|\text{gap}| > 0.1$; of the 10 large-gap runs only 4 are invalid. An agent can be wrong and honest about it, or approximately right and still misreport: *neither measurement substitutes for the other*. Both cluster in the same two substrates, but on different replicates.

Failure taxonomy, and a control arm. None of the ten unscored runs is a model capability failure: five were killed by the driver watchdog before writing a deliverable, three died on a single transient connection error, one timed out, one produced nothing usable (Appendix G). Among runs that *did* score, the substantive failures are the 15 invalid artifacts, and they split by mechanism in a way a single scalar could not: *masking* failures, NaN convention wrong and exterior inflation $2\text{--}4\times$ —the dspy pattern, whose modal masking rule is a training-derived validity mask rather than the plasma boundary polygon—and *scale* failures, mask near-perfect but magnitude wrong, the ursa pattern.

A blind, outcome-blind rubric review over three replicates per cell (7 dimensions, 1–5, median of three passes, 23 judgeable runs, \$2.53) ranks the L3 substrates pi 3.71, cursor 3.57, dspy 3.14, ursa 2.71—the frozen-score ordering up to the pi/cursor swap—and rates L3 above L2 (3.07 vs 2.86). Where it *disagrees* is instructive: it judged the adaptive sampling “genuine” in 21 of 23 runs, while the code analysis of Section 7 finds the criterion actually reacting to what was measured in 2 of 37. A reviewer reading code accepts nominal adaptivity that a parser does not. The reviewer is also the model that produced every run it grades, so its scores only corroborate the deterministic analysis (Section 8).

Table 2: What each cell chose, and whether the code agrees with the prose. *Agr.* is the share of a cell’s replicates on its modal chain of methods (method reproducibility, not score reproducibility); *Switch.* counts replicates whose acquisition criterion changed between rounds; *M-inf.* counts replicates whose point-choosing function calls the surrogate; the last column is the modal verdict of comparing the criterion each run names against the one its code computes.

Cell	Model	Acquisition	Stop	Agr.	Switch.	M-inf.	Code vs prose
L2-cursor	kernel ridge	uncertainty_gp	no thresh.	0.20	0/2	—	unverifiable 5/5
L2-dspy	kernel ridge	residual_ucb	no thresh.	0.10	1/9	10/10	partial 8, agree 2
L2-pi	kernel ridge	residual_ucb	no thresh.	0.20	1/5	—	unverifiable 5/5
L2-ursa	ridge / k.r.	uncertainty	unexpl.	0.38	0/3	2/5	partial 3, mismatch 2
L3-cursor	MLP (Torch)	uncertainty_ensemble	thresh. met	0.20	0/5	5/5	agree 4 , partial 1
L3-dspy	MLP (Torch)	uncertainty_ensemble	no thresh.	0.10	0/8	8/10	partial 9, mismatch 1
L3-pi	MLP (Torch)	uncertainty_ensemble	thresh. met	0.20	0/3	5/5	agree 3 , partial 2
L3-ursa	MLP (Torch)	uncertainty	unexpl.	0.17	0/2	4/5	partial 4, mismatch 1

Finally, the benchmark asks agents to sample adaptively, so it should say whether that is worth asking for. Forcing the pipeline’s own meta-loop to use residual-UCB acquisition against a blind space-filling append—four paired replicates, identical solve budgets, no LLM calls—gives a clean negative: aggregate accuracy improves by +1.47 points on average while **worst-cell** accuracy, pre-registered as the headline because an adaptive method should pay off where the surrogate is weakest, moves by −0.23 and wins in 2/4 (Appendix H). The two summaries point in opposite directions, exactly the failure mode the pre-registration was written to expose.

7 What the agents actually did

A frozen-set score cannot distinguish two runs that reach the same error by different reasoning—one choosing its adaptive points by ensemble disagreement and stopping because validation error crossed the threshold, another relabelling uniform sampling as “adaptive” and stopping because it ran out of rounds. We measure that difference with two extractors over each finished workspace, both **LLM-free and execution-free**: they parse the agent’s Python with the `ast` module and match role-scoped patterns, so the whole archive re-extracts identically and for nothing. Scope is load-bearing—a whole-workspace `grep` for `subprocess` is true of nearly every run—so each question is answered only from the code playing the relevant role, non-production files are excluded, and every answer carries `file:line` evidence. The vocabulary is anchored on the typed action space of the scripted levels, so every level lands in the same table and anything unmatched stays `unknown`.

Method choice tracks the access level, not the substrate. Table 2 shows a split no outcome metric reveals. **Every L2 cell’s primary model is kernel ridge** (17 of the 20 replicates with a classifiable primary)—the model the library’s own zoo also selects in all six L0 runs—and **every L3 cell’s is a hand-written PyTorch MLP** (25 of 26). Acquisition splits the same way: L2 agents gravitate to residual-UCB and GP variance, the two strategies the L2 prompt names as available in the library, while L3 agents converge on ensemble disagreement, which no prompt mentions. Four independent substrates, given a library, reproduce its defaults; given nothing, all four rediscover a different method family and two beat the library with it. That is the mechanism behind the level effect, and why we read it as anchoring rather than as the library being bad code.

Method reproducibility is poor, even where score reproducibility is good. Chain agreement never exceeds 0.38 and is 0.10 in both ten-replicate cells: the same agent, prompt and model produces a different pipeline nearly every time. **L3-cursor** is the sharpest case—five distinct chains in five replicates, while its error stayed inside [0.062, 0.212] and every artifact was valid. A benchmark reporting only the score spread concludes the agent is consistent; it is consistently *good*, but it is not doing the same thing twice.

Nominal adaptivity is one fixed rule executed n times. The task mandates up to three adaptive rounds whose points are chosen from the current model or data. The criterion actually changed between rounds in **2 of the 37** runs where it could be classified; the rest ran one rule three times. The crude failure—a stated criterion that is pure randomness— occurs in no cell; the subtler one, non-reactive adaptivity, is near-universal, which is consistent with the control arm finding little for a reactive criterion to earn at this budget.

The code often does not match the prose, and a library hides it. Comparing the criterion each run *states* against the one its code *computes* gives partial 27, unverifiable.from.code 14, agree 9 and mismatch 4 of the 54 runs with a verdict; agree is modal in only two cells, both L3, and the four mismatches fall in the two substrates that also produce every invalid artifact. Seven L2 runs name a method for which their code carries no evidence at all, against **zero in any L3 cell**. Worse for an evaluator, L2-cursor and L2-pi are unverifiable.from.code in all ten runs: selection happens inside the library, so no workspace function chooses points—a structural consequence of scaffolding, not evasion, but it means **the more scaffolding an evaluation provides, the less of the agent’s reasoning stays inspectable**. Twelve further dimensions (Appendix I) show where the rest of the solution agrees— the test set stays out of every fitting function, the pilot and storage-validation gates hold in 59/59 runs, no L3 run hand-built a mesh—and the one mandatory constraint quietly violated: roughly half the corpus satisfies the deliverable self-test by *importing predict.py* rather than re-running it in a fresh process.

8 Limitations

The full list is in Appendix F; four constrain what may be concluded. **The primary statistical test was changed after seeing the data**—we pre-specified a sign test over four harness pairs, found it cannot return $p < 0.125$ at any effect size, and moved to a stratified rank test; all three agree in sign, so the direction does not depend on the choice, but the p -value does. **L3 receives two scaffolding instructions L2 does not** (a meshing milestone and a warning against hand-built meshes), arguably necessary because L3 agents must write solver plumbing L2 agents delegate, so the contrast bundles library access with a prompt difference and is not a pure library effect. **Replicate counts are unequal and partly failure-determined**, and denominators count attempts, so L2-dspy is 10/15 where among executed runs it is 10/10—a defensible convention that moves the pooled L2 pass rate materially. And **one model, which the judge shares**: model control buys internal validity with external, the harness effect may itself be model-dependent, and the blind review of Section 6.1 is performed by the same model that produced every run it grades, so self-preference cannot be excluded and no correction is applied.

9 Conclusion

On this task, giving an agent a proven library made it worse: from-scratch agents beat library-assisted ones in all four substrates, pooled median relative L_2 0.137 against 0.229, and the deterministic analysis says why—given a library, four independent substrates reproduced its default model and its named acquisition strategies; given nothing, all four converged on a different method family and two beat the library’s own score. Holding the model fixed, the harness mattered more than the access level, and the pipeline with no language model at all still beat five of the eight agent cells. The durable point is independent of how those numbers land. A benchmark that checks only whether an agent produced the required files will silently certify work that is physically wrong and self-reported dishonestly: here 31% of scored runs passed every gate while being physically invalid, and the runs that misreported their own error were largely *not* the same runs. Separating *did it deliver, is it valid* and *did it tell the truth* costs three diagnostics computed from predictions the benchmark already holds; asking *did it do what it said* costs a parser.

Acknowledgments and Disclosure of Funding

The campaign ran on a single workstation with no external compute grant. We thank the Open FUSION Toolkit developers, whose solver and worked examples are the substrate this benchmark is built on, and the maintainers of the four agent frameworks evaluated here, none of whom were involved in or consulted about this evaluation.

References

- Lakshya A. Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziemis, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J. Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G. Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. GEPA: Reflective prompt evolution can outperform reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2026. Oral. arXiv:2507.19457.
- Jonathan Bragg, Mike D’Arcy, et al. AstaBench: Rigorous benchmarking of AI agents with a scientific research suite. *arXiv preprint arXiv:2510.21652*, 2025. Allen Institute for AI.
- Kathryn Chaloner and Isabella Verdinelli. Bayesian experimental design: A review. *Statistical Science*, 10(3): 273–304, 1995. doi: 10.1214/ss/1177009939.
- Jun Shern Chan, Neil Chowdhury, Oliver Jaffe, James Aung, Dane Sherburn, Evan Mays, Giulio Starace, Kevin Liu, Leon Maksin, Tejal Patwardhan, Lilian Weng, and Aleksander Madry. MLE-bench: Evaluating machine learning agents on machine learning engineering. In *International Conference on Learning Representations (ICLR)*, 2025.
- Harrison Chase. LangChain. <https://github.com/langchain-ai/langchain>, 2022. Software; citation metadata from the repository’s CITATION.cff, dated 2022-10-17.
- Ziru Chen, Shijie Chen, Yuting Ning, Qianheng Zhang, Boshi Wang, Botao Yu, Yifei Li, Zeyi Liao, Chen Wei, Zitong Lu, Vishal Dey, Mingyi Xue, Frazier N. Baker, Benjamin Burns, Daniel Adu-Ampratwum, Xuhui Huang, Xia Ning, Song Gao, Yu Su, and Huan Sun. ScienceAgentBench: Toward rigorous assessment of language agents for data-driven scientific discovery. In *International Conference on Learning Representations (ICLR)*, pages 12138–12194, 2025.
- B. Efron. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1):1–26, 1979. doi: 10.1214/aos/1176344552.
- Michael Grosskopf, Nathan Debardeleben, Russell Bent, Rahul Somasundaram, Isaac Michaud, Arthur Lui, Alexius Wadell, Warren D. Graham, Golo A. Wimmer, Sachin Shivakumar, Joan Vendrell Gallart, Harsha Nagarajan, and Earl Lawrence. URSA: The universal research and scientific agent, 2025. Los Alamos National Laboratory. Software: <https://github.com/lanl/ursa>.
- C. Hansen, I. G. Stewart, D. Burgess, M. Pharr, S. Guizzo, F. Logak, A. O. Nelson, and C. Paz-Soldan. Toka-Maker: An open-source time-dependent Grad-Shafranov tool for the design and modeling of axisymmetric fusion devices. *Computer Physics Communications*, 298:109111, 2024. doi: 10.1016/j.cpc.2024.109111. arXiv:2311.07719.
- Qian Huang, Jian Vora, Percy Liang, and Jure Leskovec. MLEAgentBench: Evaluating language agents on machine learning experimentation. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of *PMLR*, pages 20271–20309, 2024.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.
- Liqiang Jing, Zhehui Huang, Xiaoyang Wang, Wenlin Yao, Wenhao Yu, Kaixin Ma, Hongming Zhang, Xinya Du, and Dong Yu. DS Bench: How far are data science agents from becoming data science experts? In *International Conference on Learning Representations (ICLR)*, 2025.
- Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. DSPy: Compiling declarative language model calls into self-improving pipelines. In *International Conference on Learning Representations (ICLR)*, 2024.

- Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. AgentBench: Evaluating LLMs as agents. In *International Conference on Learning Representations (ICLR)*, 2024.
- Bodhisattwa Prasad Majumder, Harshit Surana, Dhruv Agarwal, Bhavana Dalvi Mishra, Abhijeetsingh Meena, Aryan Prakhar, Tirth Vora, Tushar Khot, Ashish Sabharwal, and Peter Clark. DiscoveryBench: Towards data-driven discovery with large language models. In *International Conference on Learning Representations (ICLR)*, 2025. Allen Institute for AI.
- M. D. McKay, R. J. Beckman, and W. J. Conover. A comparison of three methods for selecting values of input variables in the analysis of output from a computer code. *Technometrics*, 21(2):239–245, 1979.
- Ludovico Mitchener, Jon M. Laurent, Alex Andonian, Benjamin Tenmann, Siddharth Narayanan, Geemi P. Wellawatte, Andrew White, Lorenzo Sani, and Samuel G. Rodrigues. BixBench: A comprehensive benchmark for LLM-based agents in computational biology. *arXiv preprint arXiv:2503.00096*, 2025. FutureHouse.
- Burr Settles. Active learning literature survey. Computer Sciences Technical Report 1648, University of Wisconsin–Madison, 2009.
- Zachary S. Siegel, Sayash Kapoor, Nitya Nadgir, Benedikt Stroebel, and Arvind Narayanan. CORE-Bench: Fostering the credibility of published research through a computational reproducibility agent benchmark. *Transactions on Machine Learning Research (TMLR)*, 2025.
- Giulio Starace, Oliver Jaffe, Dane Sherburn, James Aung, Jun Shern Chan, Leon Maksin, Rachel Dias, Evan Mays, Benjamin Kinsella, Wyatt Thompson, Johannes Heidecke, Amelia Glaese, and Tejal Patwardhan. PaperBench: Evaluating AI’s ability to replicate AI research. *arXiv preprint arXiv:2504.01848*, 2025.
- P. H. van Elteren. On the combination of independent two-sample tests of wilcoxon. *Bulletin of the International Statistical Institute*, 37:351–361, 1960.
- Hjalmar Wijk, Tao Lin, Joel Becker, Sami Jawhar, Neev Parikh, Thomas Broadley, Lawrence Chan, Michael Chen, Josh Clymer, Jai Dhyani, Elena Elicheva, Katharyn Garcia, Brian Goodrich, Nikola Jurkovic, Holden Karnofsky, Megan Kinniment, Aron Lajko, Seraphina Nix, Lucas Sato, William Saunders, Maksym Taran, Ben West, and Elizabeth Barnes. RE-Bench: Evaluating frontier AI R&D capabilities of language model agents against human experts. *arXiv preprint arXiv:2411.15114*, 2024. METR.
- Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927. doi: 10.1080/01621459.1927.10502953.
- Yunbei Zhang, Janet Wang, Yingqiang Ge, Weijie Xu, Jihun Hamm, and Chandan K. Reddy. Stop comparing LLM agents without disclosing the harness. *arXiv preprint arXiv:2605.23950*, 2026.

A The benchmark loop

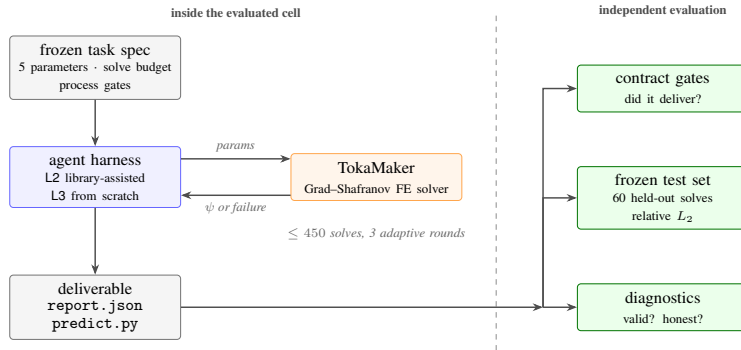


Figure 2: The benchmark loop. The agent must *buy* its own training data from an expensive Grad–Shafranov solver under a fixed budget, choosing where to sample across three adaptive rounds, and emits a deliverable that is checked, scored and diagnosed entirely outside the cell on a test set it never sees. Every condition passes through the identical right-hand column.

B Reproducibility

```
pip install -e ".[ml,dev,harnesses]"
python -m autotokamak.bench freeze-testset # ground truth (committed)

tools/campaign_guard.py preflight --harnesses "ursa dspy pi cursor" \
    --task benchmarks/tasks/L3_mini_v3.yaml
tools/run_campaign.sh --tag <tag> --reps 5 --parallel 6 --budget-usd 250 \
    --harnesses "ursa dspy pi cursor"

python tools/cost_report.py --tag <tag>
python tools/aggregate_matrix.py --tag <tag> # main table + all three tests
python tools/render_paper_figures.py --tag <tag> --out <paper>/figures
python tools/render_paper_tables.py --tag <tag> --out <paper>/tables

python tools/run_library_baseline.py --n-samples 450 --seeds 0 1 2 # the L0 arm
python tools/run_control_arm.py --reps 4 # Section 7.4
python tools/judge_code.py score --tag <tag> --judge-provider openai \
    --samples 3 --per-cell 3 # Section 7.3
```

Task texts are frozen and versioned; every run records the task’s SHA-256, its prompt version, its model string, and its replicate index. The frozen test set carries provenance attributes (solver version, parameter and grid hashes, scoring epoch) and is never regenerated. The statistics are seeded (bootstrap and both permutation tests use seed 20260917, 10 000 and 20 000 resamples respectively), so the analysis reproduces exactly from the archived runs. Every figure and appendix table in this paper is generated from those artifacts by the two scripts above rather than transcribed.

The one thing that does *not* reproduce exactly is the agent runs themselves: they are stochastic, and the substrates are versioned third-party software that moves. That is why the campaign is replicated and reported with intervals rather than as point estimates.

C Benchmark documentation

Access and hosting. The benchmark—task specifications, the deliverable contract, the frozen evaluation assets, all harness adapters and every analysis script—is at <https://github.com/hindyros/autokamak>. The results in this paper are from the campaign tagged `matrix-v4-20260919` at commit `dbc8289`. An archival DOI is to be minted at camera-ready from the release tag.

Licensing. The benchmark code and task specifications are MIT licensed. The ground-truth solver, Open FUSION Toolkit / TokaMaker [Hansen et al., 2024], is LGPL and is used as an unmodified library dependency, which MIT-licensed code may do; we redistribute no part of it. The frozen test set is derived data produced by that solver from parameter vectors we generated, and is released under the same MIT terms as the rest of the benchmark.

Maintenance. The contract, the evaluation grid and the frozen test set are versioned assets: changing any of them makes archived scores incomparable, so the policy is that they are not edited—a change means a new scoring epoch, recorded in the archive’s provenance attributes, and results are never pooled across epochs. New diagnostics are added as recorded measurements, never as gates, for the same reason. A new agent substrate is one adapter module implementing `run(task, workspace, run_dir, model, timeout)` plus one registry entry; contributions are accepted on that interface.

Author statement. The authors bear all responsibility in case of violation of rights, and confirm that the assets released with this paper are released under the license stated above.

Compute. The entire campaign ran on a single workstation (Apple silicon, macOS): 59 agent runs consuming 19.1 hours of wall clock with several cells concurrent, plus the solver time inside each run, which is CPU-bound and is the dominant cost of a cell in seconds if not in dollars. Model spend was \$197.78 in total. The two supporting arms (Section 5 and Appendix H) make no model calls at all and cost \$0.00; they take roughly 5 and 2 CPU-hours respectively. No GPUs or clusters were used, and reproducing the non-agentic parts of this paper requires neither.

Ethical considerations and broader impact. The benchmark contains no human-subject or personally identifying data. Its intended positive impact is on evaluation practice: we show that a benchmark scoring only deliverable compliance certifies physically wrong artifacts at a rate of roughly one in three, which is an argument for cheap independent validity checks in any agent evaluation whose output is a scientific artifact. The foreseeable negative is the ordinary one for capability benchmarks—that scores become a target, and that a substrate could be tuned to this task’s contract rather than to the underlying problem; the frozen, never-regenerated test set and the diagnostics that sit outside the contract are our partial defence. The dual-use surface is that of the underlying solver, which is publicly available open-source software [Hansen et al., 2024], and this work adds nothing to it.

LLM usage. Large language models are the object of study: all eight agent cells are driven by gpt-5.2 (Section 4), and the blind code review in Section 6.1 uses the same model as a judge, with the self-preference caveat recorded in Section 8. The deterministic analyses of Sections 6.1 and 7 use no language model at any point.

D Enforcing an equal compute budget

The long form of the summary in Section 4, reported because the failure modes generalise to anyone building an agent evaluation.

Fair comparison requires every substrate to get the same budget, and this was the single hardest engineering requirement in the benchmark. Two adapters originally accepted a `timeout.seconds` argument and silently ignored it. Fixing that exposed two deeper failures, both found only by running a full campaign: a timeout raised as an ordinary exception is *swallowed*, because agent frameworks wrap their step loops in broad `except` Exception handlers and `signal.alarm` is one-shot, so a single absorbed alarm disables the cap permanently—one harness ran over ten hours against a 90-minute budget; and killing the agent process does not stop the work, because `subprocess.run(capture_output=True, timeout=...)` waits again on pipes that the solver’s grandchildren still hold open, so the timeout path itself blocks—one cell sat in that deadlock for 9 hours 19 minutes. The fixes are to raise a `BaseException` subclass, for the same reason `KeyboardInterrupt` is one, and to run each agent in its own process group and signal the group.

The general lesson generalises past this benchmark: **an agent harness is third-party code and cannot be trusted to honour a deadline from inside**. Our final design enforces the budget at three independent levels—in-adapter (graceful, and still records cost and partial output), process-group kill, and a driver-side watchdog that nothing inside the cell can intercept. Benchmarks that enforce time limits only in-process silently grant unequal compute to whichever substrate is least well behaved, which is precisely backwards. We also equalised `feedback.rounds` to 1 for this campaign: only two of the four adapters implement outer review-and-fix rounds at all, so any higher value handed those two a second pass the others never got—the mechanical cause of their longer runtimes and larger spend in earlier campaigns. One is the only value every adapter honours identically.

E Task characterisation in full

From a direct sweep independent of any agent: pool $n = 9993$, test $n = 1499$. Error is reported as a ratio to the mean-predictor baseline, so 1.0 is “no better than predicting the average field”. Summarised in Section 3.2.

Table 3: Scaling of surrogate error with training-set size. “Best ratio” is relative L_2 divided by the mean-predictor baseline, minimised over the model zoo.

N	50	100	200	400	800	1600	3200	6400
best ratio	0.575	0.526	0.485	0.465	0.369	0.303	0.278	0.268
R^2	0.804	0.840	0.865	0.876	0.922	0.948	0.956	0.959

Table 4: Error ratio by sampling design at matched budget.

N	space-filling	random	clustered
100	0.550	0.699	0.963
400	0.358	0.438	0.873
1600	0.289	0.309	0.808

Table 5: Out-of-distribution generalisation. Rows in bold are the transfer directions; both collapse to the mean predictor or worse.

train $\rightarrow$ test	ratio	R^2
low $\rightarrow$ low	0.376	0.927
high $\rightarrow$ high	0.401	0.919
low $\rightarrow$ high	0.943	0.280
high $\rightarrow$ low	0.937	-0.097

F Limitations in full

- **The primary statistical test was changed after seeing the data.** We pre-specified a sign test over four harness pairs, found it cannot return $p < 0.125$ at any effect size, and moved to a stratified rank test. All three tests agree in sign, so the direction does not depend on the choice; the p -value does. All three are reported wherever the contrast appears.
- **L3 receives two scaffolding instructions L2 does not** (a meshing milestone and a warning against hand-built meshes), arguably necessary because L3 agents must write solver plumbing L2 agents delegate. The contrast bundles library access with a prompt difference and is not a pure library effect.
- **Unequal, partly failure-determined replicate counts.** Two substrates have $n = 10$ and two $n = 5$. Denominators count attempts, so L2-dspy is 10/15 where among executed runs it is 10/10; both conventions are defensible and the choice moves the pooled L2 pass rate materially, so we state ours wherever a pass rate appears.
- **One model, and the judge shares it.** Model control buys internal validity with external: the harness effect may itself be model-dependent. The blind review of Section 6.1 is moreover performed by the same model that produced every run it grades, so self-preference cannot be excluded and no correction is applied—its scores corroborate the deterministic analysis and are never a measurement in their own right.
- **Accounting caveats.** Wall-clock time is contaminated by concurrency on one CPU-bound machine and no executed-time measure is recorded, so cost and turn counts are the efficiency axes we report; one L3-ursa run was killed at a cap later runs did not face; and two substrates reuse a run identifier across sibling directories, so a join on it reuses one replicate’s cost (we report the directory-keyed \$197.78 and note the \$199.76 discrepancy). Only two of the four ladder rungs are filled at this prompt version.
- The benchmark is **fixed-boundary, single profile family, five parameters**: a real physics problem, but not a full one.

G Failure taxonomy in full

None of the ten unscored runs is a model capability failure: five L2-dspy runs were terminated by the driver watchdog before writing a deliverable (reconciled as explicit non-passing stubs rather than dropped, which is why that cell’s denominator is 15); three L2-ursa runs died after one turn on a single transient connection error, \$0.05 each, because that adapter has no retry surviving it; one L3-ursa run timed out; and one completed with 150 solves attempted, one succeeded, and no usable deliverable. Among runs that *did* score, the substantive failures are the 15 invalid artifacts, and they split by mechanism in a way a single scalar could not separate: *masking* failures, NaN convention wrong and exterior inflation $2\text{--}4\times$ —the dspy pattern, whose modal masking rule is a training-derived validity mask rather than the plasma boundary polygon—and *scale* failures, mask near-perfect but magnitude wrong, the ursa pattern. Section 7 traces both to choices in the code.

The blind code review. Full per-cell rubric scores are in `experiments/<tag>/judge_scores.csv`; the judge model, its blindness conditions and the self-preference caveat are stated in Section 6.1 and Section 8. The one run of the 24 sampled that could not be judged is the errored L2-ursa replicate, whose workspace is empty.

H Forced-action control arm: did adaptive sampling help?

The benchmark asks agents to sample adaptively, so it should say whether adaptive sampling is worth asking for. We ran the pipeline’s own meta-loop twice over with the action *forced*, so the arms differ only in how new solver calls are chosen: treatment forces residual-UCB acquisition, control a blind space-filling append. Four paired replicates, identical per-iteration solve budgets, an envelope evaluation set, no LLM calls in either arm. The pre-registered headline is **worst-cell** accuracy, not aggregate accuracy, because an adaptive method should pay off precisely where the surrogate is weakest and the aggregate can hide that.

It is a clean negative. Aggregate accuracy improves by +1.47 percentage points on average (3/4 replicates positive), but worst-cell accuracy—the quantity we said in advance we would judge it on—moves by −0.23 points and wins in only 2/4. The two summaries point in opposite directions, which is exactly the failure mode the pre-registration was written to expose. At this budget, on this task, forced adaptive acquisition does not reliably beat blind space-filling where it matters most.

I How the agents wrote it: prose, auditability and solution shape

Stated criterion versus implemented criterion. Comparing the criterion each run *states* against the one its code *computes*, at the level of the criterion family, gives `partial 27`, `unverifiable_from_code 14`, `agree 9` and `mismatch 4` of the 54 runs with a verdict; `agree` is modal in only two cells, both L3. The four outright mismatches—stated and implemented criteria with nothing in common—fall in the two substrates that also produce every invalid artifact. Separately, three L2-dspy, two L2-pi and two L2-ursa runs name a method in their README or report for which the code carries no evidence at all, against **zero such claims in any L3 cell**.

Library access removes auditability. L2-cursor and L2-pi are `unverifiable_from_code` in all ten runs, and whether their selection is model-informed is simply unknown: selection happens inside the library, so no workspace function chooses points. This is a structural consequence of giving an agent a library, not evasion—but it means the more scaffolding an evaluation provides, the less of the agent’s reasoning stays inspectable. Every L3 cell with a classifiable chooser is model-informed. Relatedly, ursa rounds largely did not record validation error against baseline at the moment of choosing (median evidence-grounding 0.33 at L2 and 0.17 at L3, against 1.0 elsewhere), so its stopping decisions cannot be falsified from its own artifacts at all.

The whole solution shape. Twelve further dimensions ask how each run solved the rest of the task (Appendix Q). Three are identical across all eight cells: the test set stays out of every model-fitting function, the pilot gate is enforced, and every run counts a solve only after re-reading its stored artifact and confirming finite values—the two process gates added after an earlier campaign held in 59/59 runs, and no L3 run hand-built a mesh, so that prohibition held 26/26. The interesting failure is the *deliverable self-test*: the task requires re-running the documented `predict.py` command in a fresh process before declaring done, and roughly half the corpus satisfies it by *importing* the module instead. Code that has not been re-run the way it is documented to be run is the precondition for most of the interface failures this benchmark has ever recorded—a mandatory constraint being partially violated in plain sight, measurable only because the question is asked of the code that plays that role.

J The scoring rule

Predictions are scored against a frozen test set of 60 parameter vectors (generated with seed 20260809 and committed; all 60 solves converged) solved once and stamped with provenance: solver version, the SHA-256 of both the parameter file and the grid, and a scoring epoch, so that a score is only ever pooled with scores computed the same way. The archive is never regenerated. The metric is

per-sample relative L_2 over the finite ground-truth points:

$$\text{err}_i = \frac{\|\hat{\psi}_i - \psi_i\|_2}{\|\psi_i\|_2}, \quad \text{masked to } \{(R, Z) : \psi_i(R, Z) \text{ finite}\}. \quad (1)$$

A NaN prediction at a finite ground-truth point is scored as a full-magnitude miss (the prediction is treated as zero), never skipped—otherwise an all-NaN prediction scores as perfect. Accuracy is quoted against the trivial mean-flux-map predictor on the same set, $100 \times (1 - \text{relL2}/\text{baseline})$, where the baseline scores 0.4933.

K Invalid artifacts that passed every gate

From a prior campaign: an agent passed **9/9 gates** at relative L_2 **0.9596**—accuracy —**94.5%**, twice as bad as predicting the mean field—because its predictor emitted zeros instead of NaN outside the boundary, at a spread ratio of 0.027. The same family recurs here: one **L3-dspy** replicate emitted *no NaN at all* against ground truth that is 80.5% NaN, and an **L2-ursa** replicate reached a prediction spread $16.2\times$ ground truth’s. All of it is invisible to the contract, and all of it is caught by three numbers computed from predictions the benchmark already has.

L The verification plane

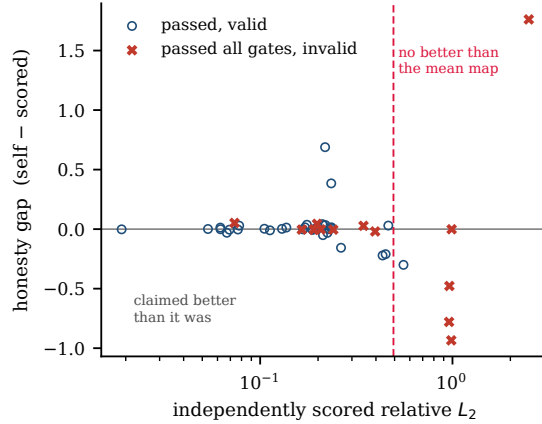


Figure 3: The verification plane. Every point passed every contract gate. Crosses are physically invalid artifacts; the dashed line is the mean-map baseline. Large honesty gaps and invalid artifacts are largely different runs.

M Cost against accuracy

N The deliverable contract in full

O Frozen assets

- `benchmarks/assets/eval_grid.json` — the frozen evaluation grid: 64 points in $R \in [0.15, 0.80]$ m and 96 in $Z \in [-0.40, 0.40]$ m, both `1inspace`. Flux arrays are stored $(N, 96, 64)$, that is (N, n_Z, n_R) .
- `benchmarks/assets/test_params.json` — the 60 held-out parameter vectors, generated with seed 20260809. SHA-256 `d10a3651...d17b480`.
- `benchmarks/assets/test_set.h5` — ground-truth flux fields, all 60 solves converged; 80.46% of grid points are NaN, outside the plasma. Provenance attributes record the stamp time, the solver version (26.6), the SHA-256 of both the parameter file and the grid, the sweep configuration and its hash, and the scoring epoch (2026-08-18). Never regenerated; every archived score is relative to it.

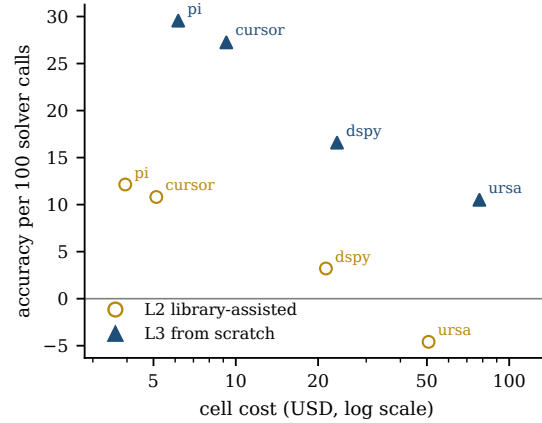


Figure 4: What each cell’s accuracy cost, at one model. A $13\times$ spread in dollars buys no consistent advantage, and the most expensive substrate is the least accurate at both access levels.

Table 6: The deliverable contract. `contract.passed` is the flat AND of all gates: no weighting, no partial credit. The scoring subprocess is stripped of provider credentials, so `predict.py` must work offline, and runs in its own process group so a timeout reaps the solver children too.

Gate	What it checks
<code>artifact:report.json</code>	present and non-empty
<code>artifact:predict.py</code>	present and non-empty
<code>artifact:README.md</code>	present and non-empty
<code>report_parsers</code>	<code>report.json</code> is valid JSON
<code>report_keys</code>	<code>n_solves_{attempted,succeeded}</code> and <code>metrics_{test,baseline}_rel_l2_{mean,median,p90}</code> present
<code>no_autotokamak_import</code>	L3 only — no workspace <code>.py</code> imports the platform library, by regex over plain imports, <code>importlib</code> , <code>__import__</code>
<code>predict_runs</code>	<code>python predict.py --input params.json --output pred.npz</code> exits 0
<code>predict_shape</code>	returned <code>psi</code> is $(N, 96, 64)$
<code>predict_grid</code>	returned <code>R, Z</code> match the frozen axes to 10^{-9}

- `benchmarks/assets/prices.json` — the dated price table behind every derived cost figure, so a published dollar amount is traceable to the rates it was computed with.
- `benchmarks/tasks/*.yaml` — the task specifications.

P The task text, and the L2/L3 difference

The L3 problem statement is reproduced verbatim below. The L2 text is byte-identical to it except for the ten hunks that follow, every one of which concerns access level: the title, the description of the solver, the three pipeline steps where the library would be used, the workspace and environment bullets, the single-OFT_{env} advice, and the constraints block. The parameter ranges, the budget, the stopping criterion, the evaluation interface, all five deliverables, the “do not fabricate metrics” paragraph and the three process gates are identical.

P.1 L3 problem statement (verbatim)

TITLE

Build, end to end, a machine-learning surrogate for fixed-boundary Grad-Shafranov tokamak equilibria | including a data-generation campaign with adaptive (active) sampling | from scratch.

THE TASK

TokaMaker, part of the pip-installed ‘OpenFUSIONToolkit’ Python package, solves the Grad-Shafranov equation for the poloidal flux $\psi(R, Z)$ of a tokamak plasma. You will use it as your ground-truth physics solver.

Consider fixed-boundary equilibria of D-shaped plasmas described by 5 input parameters, with TokaMaker's default pressure/current profile shapes:

```
r0 : major radius      [m]   in [0.35, 0.55]
a  : minor radius      [m]   in [0.10, 0.20]
kappa : elongation     [-]   in [1.0, 1.6]
delta : triangularity  [-]   in [0.0, 0.4]
Ip  : plasma current   [A]   in [80e3, 200e3]
(plasma centered at z0 = 0)
```

Your job, in full:

1. Learn how to drive TokaMaker for fixed-boundary solves (mesh a D-shaped last-closed-flux-surface, solve, extract psi). Choose your own numerical resolution; a single solve should take seconds, not minutes.
2. Design and run a DATA-GENERATION CAMPAIGN structured as an initial space-filling design of AT MOST 150 solves followed by UP TO 3 ADAPTIVE (ACTIVE-SAMPLING) ROUNDS of EXACTLY 100 new solver runs each (failed/unconverged solves count against a round's 100). A one-shot space-filling or random design alone is NOT sufficient: each round's 100 new solver evaluations must be chosen ADAPTIVELY, informed by the current model or data (your strategy | e.g. uncertainty, model error, disagreement, anything you can justify). STOPPING CRITERION: keep a held-out VALIDATION set (separate from the final test set of step 4, which must never influence the campaign); after each round, retrain your surrogate and stop the campaign early once its validation error is reduced by at least 70% relative to the mean-predictor baseline (a model that always predicts the training-set mean psi map), i.e. $\text{val_error} \leq 0.30 * \text{baseline_error}$. Otherwise continue to the 3-round cap. Log every acquisition decision (which point, when, why) and each round's validation error vs baseline to an acquisition log file.
3. Train an ML surrogate mapping the 5 parameters to the PHYSICAL poloidal flux $\psi(R, Z)$ in Webers (not a normalized flux). Model family, input/output representation, and training procedure are entirely your choice.
4. Evaluate honestly on a held-out test set of AT LEAST 20 solver runs at uniformly random parameters (fixed recorded seed), generated independently and never used for training, model selection, or acquisition.
5. Report results and analyze whether your adaptive sampling actually helped (e.g. versus the model trained on the initial design only, or any comparison you can defend).

EVALUATION INTERFACE (fixed, so results are comparable across methods)

Predictions and metrics are defined on this fixed rectangular grid:

```
R : 64 points, uniform in [0.15, 0.80] m
Z : 96 points, uniform in [-0.40, 0.40] m
```

psi arrays are shaped (96, 64) = (nZ, nR). Grid points outside the plasma boundary carry NaN; all metrics are computed only where the ground truth is finite.

Required metric: per-sample relative L2 error

```
err_i = ||psi_pred_i - psi_true_i||_2 / ||psi_true_i||_2
```

over the finite grid points. Report mean, median, and 90th percentile over the test set, and the same numbers for a trivial baseline (predict the training-set mean psi) so the surrogate's value is evident.

DELIVERABLES (all in your current working directory; internal file formats and code organization are your choice)

1. Your pipeline code, runnable end to end by one documented entry command.
2. The dataset(s) you generated, plus the acquisition log.
3. The trained model artifact, plus a prediction script with EXACTLY this interface:

```
python predict.py --input params.json --output pred.npz
```

 where params.json is a JSON list of objects with keys r0, a, kappa, delta, Ip, and pred.npz contains:

```
psi : (N, 96, 64) float64 predicted flux [Wb], NaN outside plasma
R   : (64,) float64      grid R coordinates [m]
Z   : (96,) float64      grid Z coordinates [m]
```
4. report.json | machine-readable summary with at least:

```
n_solves_attempted, n_solves_succeeded, n_train, n_test,
sampling_strategy (short text), acquisition_log (path),
metrics: {test_rel_l2: {mean, median, p90},
          baseline_rel_l2: {mean, median, p90}},
adaptive_vs_initial (your evidence that adaptivity helped,
```

or an honest statement that it did not).
5. README.md | your design, why you chose it, how to run, results.

You MUST actually run the full campaign and training within this session and verify every deliverable exists before declaring done. Report failures honestly in report.json/README; do not fabricate metrics.

ENVIRONMENT FACTS

- * Your current working directory IS your workspace. Create all files here. Do not read, copy, or import code from the surrounding repository. In particular the Python environment contains an installed package called 'autotokamak' that solves a related problem | importing it, reading its source, or reusing its artifacts is FORBIDDEN: this is a from-scratch capability test, and only 'OpenFUSIONToolkit' may be used for physics.
- * 'python' resolves to a Python 3.11 venv that already has: numpy, scipy, h5py, matplotlib, pyyaml, scikit-learn, torch, optuna, OpenFUSIONToolkit. Nothing may be installed.
- * A read-only './OpenFUSIONToolkit/' symlink to the full OFT source tree is present in your workspace. BEFORE writing any solver or meshing code, study the worked TokaMaker examples under 'OpenFUSIONToolkit/src/examples/TokaMaker/' (the 'fixed_boundary/' example is directly relevant) and follow the workflow they demonstrate rather than inventing your own solver-facing plumbing.
- * TokaMaker limitation: only ONE 'OFT_env' may ever be created per Python process. Structure your code so the environment object is created once and reused across all solves in a process, or run solves in child processes.

CONSTRAINTS

- * MESHING MILESTONE (mandatory, before any campaign code): locate the fixed-boundary example under 'OpenFUSIONToolkit/src/examples/TokaMaker/', reproduce its mesh-and-solve workflow as a standalone script in your workspace, run it, and verify it produces finite psi values. Only then generalize it to your parameterized D-shape. Do not design your own meshing approach when the examples demonstrate one.
- * Do NOT construct the computational mesh yourself (e.g. via scipy Delaunay) and pass raw triangles to the solver | TokaMaker validates mesh topology and will reject hand-built triangulations. Use the meshing route the OFT examples use.
- * PILOT GATE (mandatory): before the campaign, run a pilot of at least 20 solves at random parameters. If the success rate is below 50%, STOP, diagnose the per-failure error messages, fix, and re-run the pilot. Never start the 100-solve rounds while the pilot is failing.
- * STORAGE VALIDATION GATE (mandatory): a solver run may be counted as successful ONLY after its stored artifact has been re-loaded from disk and verified: the saved psi grid must contain a nonzero fraction of finite values. Apply this check to every batch you write. A stored all-NaN psi grid is a FAILED solve regardless of solver exit status, and your reported success counts must be based on validated stored artifacts, not solver return codes.
- * DELIVERABLE SELF-TEST (mandatory, before declaring done): in a fresh Python process, run the exact documented command for each deliverable | including 'python predict.py --input <sample params> --output <out>' | and verify the output loads with the required shapes and finite values. If any deliverable fails its own interface test, fix it and re-test. Code you have not re-run after your last edit does not count as verified.
- * Do not run 'pip install' or modify the environment.
- * Do not call 'git'.
- * Do not prompt for user input; the task is fully autonomous.
- * If errors arise, debug, fix your code, and re-run. Stay within the campaign structure above (at most 3 adaptive rounds x 100 solves, plus the initial design and validation/test sets) across ALL attempts that reach the solver.

P.2 Diff from the L2 statement

```
--- L2_mini_v3
+++ L3_mini_v3
@@ -4,2 +4 @@
- campaign with adaptive (active) sampling | using the installed
- 'autotokamak' platform library wherever it helps.
+ campaign with adaptive (active) sampling | from scratch.
@@ -10,2 +9,2 @@
```

```

- psi(R, Z) of a tokamak plasma. It is your ground-truth physics
- solver, and the installed 'autotokamak' package already wraps it.
+ psi(R, Z) of a tokamak plasma. You will use it as your ground-truth
+ physics solver.
@@ -25,4 +24,3 @@
-
- 1. Drive the solver for fixed-boundary solves. You may use
-   'autotokamak.data.sweep.run_sweep' (config-driven sweeps; also
-   accepts an explicit (N, 5) sample matrix), or go through
-   'autotokamak.core' directly. A single solve should take
+ 1. Learn how to drive TokaMaker for fixed-boundary solves (mesh a
+   D-shaped last-closed-flux-surface, solve, extract psi). Choose
+   your own numerical resolution; a single solve should take
@@ -36,13 +34,13 @@
-   chosen ADAPTIVELY, informed by the current model or data
-   ('autotokamak.data.acquire' implements residual-UCB and PCA-GP
-   variance acquisition you may reuse, or bring your own).
-   STOPPING CRITERION: keep a held-out VALIDATION set (separate
-   from the final test set of step 4, which must never influence
-   the campaign); after each round, retrain your surrogate and
-   stop the campaign early once its validation error is reduced
-   by at least 70% relative to the mean-predictor baseline (a
-   model that always predicts the training-set mean psi map),
-   i.e. val_error <= 0.30 * baseline_error. Otherwise continue to
-   the 3-round cap. Log every acquisition decision (which point,
-   when, why) and each round's validation error vs baseline to an
-   acquisition log file.
+   chosen ADAPTIVELY, informed by the current model or data (your
+   strategy | e.g. uncertainty, model error, disagreement,
+   anything you can justify). STOPPING CRITERION: keep a held-out
+   VALIDATION set (separate from the final test set of step 4,
+   which must never influence the campaign); after each round,
+   retrain your surrogate and stop the campaign early once its
+   validation error is reduced by at least 70% relative to the
+   mean-predictor baseline (a model that always predicts the
+   training-set mean psi map), i.e. val_error <= 0.30 *
+   baseline_error. Otherwise continue to the 3-round cap. Log
+   every acquisition decision (which point, when, why) and each
+   round's validation error vs baseline to an acquisition log
+   file.
@@ -51,4 +49,2 @@
-   'autotokamak.surrogate' provides dataset loading, PCA
-   reduction, a model zoo (GP, kernel ridge, polynomial ridge,
-   MLP), and Optuna search; model family and representation
-   remain entirely your choice.
+   Model family, input/output representation, and training
+   procedure are entirely your choice.
@@ -107 +103,6 @@
-   here.
+   here. Do not read, copy, or import code from the surrounding
+   repository. In particular the Python environment contains an
+   installed package called 'autotokamak' that solves a related
+   problem | importing it, reading its source, or reusing its
+   artifacts is FORBIDDEN: this is a from-scratch capability test,
+   and only 'OpenFUSIONToolkit' may be used for physics.
@@ -110,8 +111,8 @@
-   optuna, OpenFUSIONToolkit, and the 'autotokamak' platform library
-   (importable; using it is ALLOWED and RECOMMENDED | that is the
-   point of this condition). Nothing may be installed.
-   * Useful autotokamak entry points: 'autotokamak.data.sweep.run_sweep'
-   (sweep or explicit-matrix solves → HDF5),
-   'autotokamak.data.acquire' (acquisition strategies),
-   'autotokamak.surrogate.dataset/reduce/zoo/optuna_search/automl_loop'
-   (loading, PCA, model factories, search). Read their docstrings.
+   optuna, OpenFUSIONToolkit. Nothing may be installed.
+   * A read-only './OpenFUSIONToolkit/' symlink to the full OFT
+   source tree is present in your workspace. BEFORE writing any
+   solver or meshing code, study the worked TokaMaker examples
+   under 'OpenFUSIONToolkit/src/examples/TokaMaker/' (the
+   'fixed_boundary/' example is directly relevant) and follow the
+   workflow they demonstrate rather than inventing your own
+   solver-facing plumbing.
@@ -119,4 +120,3 @@
-   Python process. 'autotokamak.core.solver.make_solver' handles
-   reuse via its 'env=' argument; if you bypass autotokamak, create
-   the environment object once and reuse it, or run solves in child
-   processes.
+   Python process. Structure your code so the environment object is
+   created once and reused across all solves in a process, or run
+   solves in child processes.
@@ -124,0 +125,11 @@
+   * MESHING MILESTONE (mandatory, before any campaign code): locate

```

```

+ the fixed-boundary example under
+ 'OpenFUSIONToolkit/src/examples/TokaMaker/', reproduce its
+ mesh-and-solve workflow as a standalone script in your workspace,
+ run it, and verify it produces finite psi values. Only then
+ generalize it to your parameterized D-shape. Do not design your
+ own meshing approach when the examples demonstrate one.
+ * Do NOT construct the computational mesh yourself (e.g. via scipy
+ Delaunay) and pass raw triangles to the solver | TokaMaker
+ validates mesh topology and will reject hand-built
+ triangulations. Use the meshing route the OFT examples use.
@@ -147,2 +157,0 @@
- * Do not modify the installed autotokamak/OpenFUSIONToolkit
- packages; use them as libraries.

```

Q Solution-shape cross comparison

Twelve questions, each answered from the code that plays the relevant role, with `file:line` evidence recorded per run. Cells are abbreviated by level and harness: 2c is L2-cursor, 3u is L3-ursa, and so on. A dash means the dimension is not answerable for that cell, which at L2 generally means the library owns that part of the pipeline.

Dimension	Cells	Modal value
code_shape	2c, 2p	single_script
	2d	14_modules
	2u	15_modules
	3c	8_modules
	3d	11_modules
	3p	9_modules
	3u	7_modules
entry_point	2c, 2p, 3p	2_runnable_scripts
	2d, 3u	10_runnable_scripts
	2u	9_runnable_scripts
	3c	3_runnable_scripts
	3d	7_runnable_scripts
grid_mapping	2c, 2d, 2p, 2u	—
	3c, 3d, 3p, 3u	solver_field_eval
mask_rule	2c, 2p, 3c, 3d, 3p	lcfs_polygon
	2d, 2u	training_valid_mask
	3u	lcfs_polygon+training_valid_mask
mesh_route	2c, 2u, 3c, 3d, 3p, 3u	oft_gs_domain
	2d, 2p	—
oft_env_strategy	2c, 2d, 2p, 2u	—
	3c, 3d, 3p, 3u	singleton_reused
self_test	2c, 2d, 2u, 3p, 3u	subprocess_reruns_predict
	2p, 3c, 3d	imported_not_subprocessed
solve_isolation	2c, 2d, 2p, 2u, 3c, 3d, 3u	in_process_serial
	3p	subprocess_per_batch
storage	2c, 2p, 3c, 3p	npz_per_solve+pickle
	3d, 3u	npz_per_solve+pickle+index
	2d	npz_per_solve+hdf5
	2u	npz_per_solve+hdf5+index
leakage_guard	2c, 2d, 2p, 2u, 3c, 3d, 3p, 3u	test_absent_from_fitting_functions
pilot_gate	2c, 2d, 2p, 2u, 3c, 3d, 3p, 3u	enforced
storage_validation	2c, 2d, 2p, 2u, 3c, 3d, 3p, 3u	reload_and_check_finite

R Glossary of canonical values

Every code printed in Table 2 and Appendix Q. A code a reader has to guess at is not a measurement.

`N_modules`

The pipeline is spread over N source files, none of them dominant; contrast `single_script`.

`N_runnable_scripts`

N files are runnable as programs. More than one means the documented single command is really a sequence.

`adaptive_in_name_only`

Every stated criterion names nothing but randomness: the round was labelled adaptive and was not.

agree
The criterion stated in the log/report is the one the code computes.

chain_agreement
Share of a cell's replicates that chose the modal chain of methods — method reproducibility, which is not the same as score reproducibility.

criterion_switched
The acquisition criterion CHANGED between rounds — logic reacting to what it measured, rather than one fixed rule executed n times.

deep_ensemble
Several independently initialised models, averaged.

enforced
A pilot runs and its success rate is compared against a threshold before the campaign proceeds — the task's 50% pilot gate.

evidence_grounded
The round's validation error against baseline was recorded, so its choice can be checked against what was known at the time. An ungrounded round's reasoning is unfalsifiable.

feasibility
Candidate choice weighted by whether solves there are expected to converge, steering away from regions with failed solves.

gradient_sensitivity
Points where the response is steep or curving — a sensitivity or Jacobian-driven criterion.

hdf5
One HDF5 file holding many samples. Scales best and carries attributes/provenance, at the price of concurrent-write care.

imported_not_subprocessed
predict.py is referenced but only imported or described, never run as the documented command. Code that works in the session's process and fails in a clean one is exactly what this misses.

in_process_serial
Solves run one after another inside the driving process. Simplest, and the slowest: no parallelism, and one bad solve can poison the shared solver state.

index
An index/manifest (CSV or JSONL) sits beside the arrays, listing every sample and its status. What makes a campaign resumable and auditable rather than a directory to be re-scanned.

kernel_ridge
Kernel ridge regression — a GP's mean without its variance.

lcfs_polygon
A grid point is inside the plasma iff it lies within the LCFS polygon — point-in-polygon against the D-shape boundary, whatever the helper is called (contains_points, _points_in_poly, inside_lcfs_mask). Geometric, per sample, and independent of the surrogate's own output.

lhs
Latin hypercube: stratified in every dimension at once.

mc_dropout
Dropout left active at inference and sampled repeatedly — an ensemble's spread at one model's training cost.

mismatch
The stated criterion and the implemented one have nothing in common. The strongest available signal that a run's account of itself is wrong.

mlp_torch
A hand-written PyTorch MLP. The most common choice in this corpus, usually mapping 5 parameters to PCA coefficients.

model_informed
The function that chooses points actually calls the surrogate. A model-derived criterion that never does is not one, whatever it is named.

npz_per_solve
One compressed .npz per solved sample.

oft_gs_domain
Meshed through OFT's own API (gs.Domain / define_region / build_mesh) as the worked examples do. What the task's meshing milestone asks for, and what TokaMaker's topology checks accept.

partial
 Stated and implemented criteria overlap but do not match: one side names a component the other never does — see claimed-but-not-computed.

pickle
 Python-pickled objects (pickle/joblib/torch.save) — usually the model artifact rather than the dataset.

prose_only
 A method named in the README or report.json that the code never evidences.

random
 Points drawn uniformly at random. As the CANDIDATE POOL this is normal and harmless; as the selection criterion it means the round was adaptive in name only.

reload_and_check_finite
 A function both re-loads a written artifact and checks its finite fraction — the task’s storage-validation gate, actually implemented. This is the check that catches an all-NaN grid stored as a success.

residual_ucb
 Points are targeted where the model is MEASURABLY wrong — an error model fit on residuals or out-of-fold error, often with a UCB trade-off between exploiting known weakness and exploring. Uses measurement rather than the model’s self-assessment, which can be confidently wrong.

round_cap
 Stop because the 3-round cap was reached.

single_script
 The whole pipeline is one or two files.

singleton_reused
 One OFT_env is built once and handed out on every call — a cached module global, a class guarding construction, or an lru.cache. Honours OFT’s one-env-per-process limit in the simplest way, at the cost of keeping every solve in one process: a solver crash takes the campaign with it.

sobol
 A Sobol low-discrepancy sequence — quasi-random, extensible.

solver_field_eval
 psi is read on the frozen 64x96 grid through the solver’s own finite-element interpolator (get_field_eval). Evaluates the FE basis directly, so no second interpolation error is introduced.

space_filling
 Purely geometric coverage of the input box: farthest-point/maximin, distance to the existing training set, k-means. Needs no model at all — which makes it robust, and makes it NOT adaptive in the sense of reacting to what was learned.

stop_threshold_met
 The campaign ended with the task’s 70% criterion satisfied — the intended way to finish early.

stop_unexplained
 The campaign ended and no per-round validation error was recorded, so why it stopped cannot be read from the artifacts at all.

stop_without_threshold
 The campaign ended although validation error had NOT reached 0.30x baseline: the round cap, a budget, or a timeout ended it, not the stopping rule.

subprocess_per_batch
 The driver shells out (usually to its own script) to run a batch. Insulates solver state between batches and survives a hard crash; pays process start-up and serialises through files.

subprocess_reruns_predict
 Something in the workspace invokes predict.py as a subprocess with the contract CLI — the task’s deliverable self-test, in a fresh process, which is the only way to catch an import-order or global-state dependency.

test_absent_from_fitting_functions
 No function that calls .fit()/backward()/optimizer.step() references a test path or array. Necessary, not sufficient: the task also forbids the test set influencing model SELECTION and ACQUISITION, which this check cannot see.

top_k
 The highest-scoring candidates are taken (argsort/topk).

training_valid_mask
 The predictor masks with a valid-pixel mask derived from the training data (e.g. pixels finite in

training). Cheap and stable, but the mask cannot adapt to a geometry unlike those seen in training — the same family of defect as the run that passed 9/9 gates while predicting plasma in vacuum.

uncertainty
Points are ranked by the model’s own predictive spread, without the record saying where that spread comes from. The family term; the two entries below are its specific forms.

uncertainty_ensemble
Spread across an ENSEMBLE of models (deep ensemble, bootstrap, or MC-dropout samples) — ‘query by committee’. Needs no probabilistic model, and its quality depends entirely on the members disagreeing for real rather than sharing an initialisation.

uncertainty_gp
Posterior variance of a Gaussian process (or an acquisition built on it, e.g. expected improvement). Principled and calibrated where the GP’s kernel assumptions hold; costly as the dataset grows.

uniform_random
Independent uniform draws, with no stratification.

unverifiable_from_code
No function that chooses points could be classified, so the stated criterion can be neither confirmed nor contradicted.

val_threshold_70pct
The task’s own rule: stop once validation error is at most 0.30x the mean-predictor baseline, i.e. a 70% reduction.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [\[Yes\]](#)

Justification: The abstract and Section 1 state exactly three claims—that from-scratch agents outperform library-assisted ones on this task, that the harness matters more than the access level with the model held fixed, and that contract gates certify physically invalid artifacts—and Sections 6–8 supply the measurement for each. Scope is stated narrowly: one task, one model, four substrates.

Guidelines:

- The answer [\[N/A\]](#) means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [\[No\]](#) or [\[N/A\]](#) answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [\[Yes\]](#)

Justification: Section 9 is a dedicated limitations section. It discloses the post-hoc change of primary statistical test, the unequal and partly failure-determined replicate counts, the single-model design, the prompt difference bundled into the L2/L3 contrast, judge self-preference, a mid-campaign timeout policy change and a cost-accounting defect.

Guidelines:

- The answer [\[N/A\]](#) means that the paper has no limitation while the answer [\[No\]](#) means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.

- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: The paper contains no theoretical results.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: Section 5 and Appendix A give the full protocol; Appendix E reproduces the task text verbatim and the complete L2/L3 diff; Appendix D lists every contract gate; Appendix C documents the frozen assets and their hashes. Every seed is recorded and the statistics are seeded. Agent runs are stochastic by nature, which is why the campaign is replicated and reported with intervals.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.

- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: The benchmark, the task specifications, the frozen evaluation assets, all harness adapters and every analysis script are released at the repository named in Appendix B under the MIT licence, at the commit stated there. The exact commands that produce every table and figure in the paper are in Appendix A.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: Section 5 specifies the model and its per-adapter pinning, the harness versions, the budget enforcement, the equalised feedback-round setting and the analysis plan. The train/validation/test split policy is part of the frozen task text (Appendix E); the held-out test set is fixed, external to the agent, and described in Section 3.3.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: Pass rates carry Wilson score intervals and relative- L_2 medians carry percentile bootstrap intervals (10 000 resamples, stated seed). The level contrast is reported with three tests—a stratified rank test, a median-difference permutation test and an exact sign test—and the permutation p is reported at its resolution floor rather than as 0. The change of primary test is disclosed as post-hoc in Sections 5 and 9.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: Appendix B: a single workstation, no GPUs or cluster, 59 agent runs over 19.1 hours of wall clock with cells run concurrently, and \$197.78 of model spend. Per-cell wall-clock medians and per-harness spend are in Section 6.5. Wall clock is explicitly not offered as an efficiency metric, because cells were concurrent on one CPU-bound machine.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: The research involves no human subjects, no personal data and no scraped content. It evaluates software agents on a numerical physics task, and conforms to the NeurIPS Code of Ethics in every respect we are aware of.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: Appendix B discusses both: the intended positive impact on evaluation practice—cheap validity checks catch a failure rate of roughly one in three that compliance checking misses—and the foreseeable negative, that a benchmark score becomes a target and a substrate could be tuned to the contract rather than the problem.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The paper releases no models, no scraped data and no generative assets. The only data released is 60 solved plasma equilibria produced by a publicly available open-source solver; it carries no misuse risk beyond that solver, which we do not redistribute.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: The ground-truth solver is cited and its LGPL licence and library-only use are stated in Appendix B. All four evaluated agent substrates are cited with the exact versions used (Section 5), as are the optimiser and typed-module libraries the pipeline depends on. No asset is used outside its terms.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.

- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
 - If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
 - For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
 - If this information is not available online, the authors are encouraged to reach out to the asset’s creators.
13. **New assets**
 Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?
 Answer: [Yes]
 Justification: The new assets are the benchmark itself—task specifications, deliverable contract, frozen evaluation grid and test set, harness adapters and analysis tooling. Appendices B and C document access, licensing, maintenance policy, provenance attributes and the versioning rule that makes archived scores comparable.
 Guidelines:
- The answer [N/A] means that the paper does not release new assets.
 - Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
 - The paper should discuss whether and how consent was obtained from people whose asset is used.
 - At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.
14. **Crowdsourcing and research with human subjects**
 Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?
 Answer: [N/A]
 Justification: The paper involves no crowdsourcing and no human subjects.
 Guidelines:
- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
 - Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
 - According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.
15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**
 Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?
 Answer: [N/A]
 Justification: The paper involves no human subjects, so no IRB or equivalent review was required.
 Guidelines:
- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
 - Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
 - We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
 - For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.
16. **Declaration of LLM usage**

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [\[Yes\]](#)

Justification: Large language models are the object of study, not an incidental writing aid: every one of the eight agent cells is driven by a pinned language model (Section 5), and the blind code review of Section 7.3 uses a language model as a judge, with the resulting self-preference confound recorded in Section 9. The validity diagnostics of Section 7.1 and the method extraction of Section 8 use no language model at any point.

Guidelines:

- The answer [\[N/A\]](#) means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.